



Technical Test – Full-Stack developer (Frontend oriented)

Dear candidate,

As part of the recruitment process at Bagira Systems, you are required to complete this assignment. You will receive the assignment at the time of your choice (e.g., 9 AM) and must submit your solution within 24 hours (e.g., by 9 AM the next day) ***please read the description carefully before starting.***

Objectives:

Implement missing parts in a provided code skeleton (backend and frontend) for a small web application used to create training scenarios and manage entities inside each scenario

Instruction:

1. Clone this repository – [Bagira-Assignment](#) and create a fork with the following structure – ‘BagiraAssignment_<YourName>’
2. Make sure to work on the folder named ‘**Frontend-Oriented**’.
3. The Repo contains frontend skeleton for React and Angular implementation – choose one to implement and specify which one you chose.
4. Follow the **README** instructions on the backend and frontend folders for detailed instructions.
5. Implement the missing functionality marked in the code (TODOs) and ensure the app works end-to-end.

Requirements:

1. Data rules:
 - EntityType must be one of the following - Soldier, Tank, Drone, Aircraft, Vehicle, Civilian.
 - TaskForce must be one of the following - Friendly, Enemy.
 - Latitude must be between -90 and 90.
 - Longitude must be between -180 and 180.
 - Entities always belong to exactly one Scenario (no orphan entities).
2. Screens:
 - Scenario List – view all scenarios, navigate to details, create.
 - Create Scenario – the form should contain name and description.
 - Scenario Details – view scenario info and manage its entities.
 - Create Entity – the form must include:
 - EntityType
 - TaskForce
 - Name/Callsign
 - Latitude
 - Longitude
3. Backend Features (partial backend provided):
 - The following are already provided:
 - Domain models (Scenario, Entity, EntityType, TaskForce).
 - Repository interfaces (IScenarioRepository, IEntityRepository).
 - DTO structure (request/response DTOs).
 - Controller stubs (ScenariosController, EntitiesController) – method signatures are defined, bodies are empty.
 - You must implement:
 - Data Access Layer – implement ScenarioRepository and EntityRepository (stubs are provided). Choose In-Memory or Database.
 - If using a database: set up DbContext, migrations, and connection strings.
 - API Controllers – implement all endpoint bodies in ScenariosController and EntitiesController.
 - CORS – configure in Program.cs to allow your frontend origin.
 - DTO Validation – add validation attributes to request DTOs.
 - Error Handling – proper HTTP status codes and ErrorResponse for validation errors.
 - Validation – server-side data validation based on the data rules above.
4. Frontend (build from scratch):
 - Build a new frontend application using React or Angular (or any framework you prefer, your choice – specify which).



- Implement all required screens (Scenario List and Create, Scenario Details, Create Entity).
 - Connect all frontend services to the backend APIs.
 - Implement filtering and sorting for scenarios and entities.
 - Implement a global search feature (search across scenarios and entities).
 - Implement a map view for entities using their coordinates (latitude/longitude).
 - Implement loading states, empty states, and error handling throughout.
 - Implement toast notifications for success and error feedback.
 - Ensure client-side and server-side validation errors are displayed to the user.
5. Docker:
- After implementing the core functionality, containerize the solution using Docker:
 - i. Create Docker files for Backend and Frontend
 - ii. Create a docker-compose.yml.
 - iii. Ensure the solution runs with: docker-compose up
 - iv. Update the main README with Docker instructions
 - Notes:
 - i. Use multi-stage builds for frontend (build + serve)
 - ii. Configure CORS in backend to allow frontend origin
 - iii. Use environment variables for API base URL in frontend
 - iv. Document any required environment variables
6. Data persistency:
- **Bonus:** If you have time, connect the server to a DB of your choosing
7. Optional bonus:
- Add tests for validation (backend) and key UI flows (frontend).
 - Implement Update and Delete capabilities end-to-end for Scenarios and Entities controller.

Acceptance criteria:

- User can create a scenario and see it in the scenario list.
- User can open scenario details and view entities belonging to that scenario only.
- User can select TaskForce (Friendly/Enemy) when creating entities, and TaskForce is displayed in entity lists.
- User can filter and sort entities by Type, TaskForce, and other fields.
- User can search for scenarios and entities.
- User can view entities on a map and toggle between table and map views.
- Toast notifications appear for successful operations and errors.
- Frontend validates all input fields (coordinates, types, required fields).
- Solution runs locally.
- Solution runs in Docker using docker-compose up with all services properly configured and communicating.
- Using clean architecture.
- Code quality and maintainability (structure, naming, separation of concerns).
- Validation and error handling.
- Reasonable scope management within the timebox.

Please provide:

- A link to your branch.
- Short notes (bullet points) describing what you completed and any trade-offs.

Good Luck!